

UniCaronas

Sistema de Caronas Universitarias

Resumo da Reuniao de Planejamento Sprint 8 Final

Reta Final de Desenvolvimento e Publicacao

Engenharia de Software — 3ESCN

Periodo: 15/05/2026 a 21/05/2026 (1 semana)

Funcao	Nome
Product Owner — define e esclarece os requisitos	Rafael Machado
Scrum Master — facilita a cerimonia	Ariane da Silva Archanjo
Dev Team — estima, planeja e se compromete	Pedro Kafka, Matheus Sizanowski, Rafael Machado, Ariane da Silva Archanjo

1. Visao Geral do Sprint

1.1 Contexto e Objetivo

O Sprint 8 representa a reta final do desenvolvimento do UniCaronas — plataforma de caronas universitarias projetada para conectar estudantes de forma segura, economica e sustentavel. Esta e a etapa de encerramento tecnico do projeto, dedicada a estabilizacao do sistema, resolucao de debitos tecnicos acumulados, refinamento de experiencia de usuario, endurecimento de seguranca e preparacao completa do ambiente para publicacao.

Apos sete sprints progressivos que entregaram desde o core de autenticacao e gerenciamento de caronas ate a validacao por e-mail institucional, dashboard de impacto (Eco-Stats) e exportacao de comprovantes em PDF, o sistema encontra-se funcionalmente completo em sua espinha dorsal. Contudo, a transicao para producao exige uma sprint dedicada exclusivamente a qualidade, robustez e estabilidade — o que define o Sprint 8 Final.

1.2 Importancia da Etapa Final

Sprints finais de projetos academicos de software carregam uma responsabilidade dupla: demonstrar maturidade tecnica da equipe e entregar um sistema que funcione de forma consistente em um ambiente proximo ao real. O Sprint 8 nao introduz novas funcionalidades de negocio — seu foco e garantir que tudo o que foi construido funcione com qualidade de producao.

Os principais pilares desta sprint sao: (1) finalizacao das funcionalidades entregues parcialmente, (2) eliminacao dos debitos tecnicos identificados no SPRINT_STATUS.md, (3) fortalecimento de seguranca documentado no SEGURANCA.md, (4) otimizacao de performance e UX, (5) cobertura de testes abrangente e (6) preparacao completa do deploy.

1.3 Dados da Cerimonia de Planejamento

Campo	Detalhe
Data	14/05/2026 (quinta-feira)
Formato	Presencial
Duracao	Aproximadamente 1h30
Quem conduziu	Ariane da Silva Archanjo (Scrum Master)
Quem definiu as tarefas	Rafael Machado (Product Owner)
Inicio da Sprint	15/05/2026
Fim da Sprint	21/05/2026 (1 semana)
Carater	Sprint Final de Encerramento — correcao de debitos tecnicos, endurecimento de seguranca, finalizacao de funcionalidades parciais e preparacao para deploy/entrega

2. O que sera entregue ao final do Sprint

Ao final deste Sprint, o UniCaronas estara pronto para publicacao e apresentacao final com as seguintes entregas consolidadas:

- Rate Limiting implementado nas rotas criticas (login, cadastro, API publica) — protecao contra ataques de forza bruta
- CORS restringido ao dominio real do frontend em modo producao — eliminacao do wildcard permissivo
- Tokens JWT migrados para HttpOnly Cookies — eliminacao da vulnerabilidade XSS no localStorage
- Protecao CSRF implementada para rotas POST/PATCH/DELETE sensiveis
- Recuperacao de senha completamente funcional com envio real de e-mail via Nodemailer
- Visualizacao de mapa integrada com Leaflet.js nas telas de criacao e detalhes de carona
- Rate Limiting e headers de seguranca revisados e endurecidos
- Console.log de producao eliminados — logging profissional implantado
- Race Condition na lista de espera corrigida com bloqueio otimista no banco
- Frontend modularizado — arquivos JS grandes quebrados em modulos coesos
- Painel Admin expandido com metricas de uso e graficos basicos
- Testes automatizados com cobertura $\geq 70\%$ nos servicos criticos
- Checklist de deploy completo validado — variaveis de ambiente, build, logs, backup
- README.md final atualizado com todas as variaveis de ambiente do Sprint 7 e Sprint 8

A entrega esta completa quando:

- Todas as vulnerabilidades listadas em SEGURANCA.md estao corrigidas ou mitigadas
- Testes automatizados passam com cobertura documentada $\geq 70\%$ nos modulos criticos
- Sistema executa sem erros criticos no fluxo completo: cadastro -> verificacao de e-mail -> login -> buscar carona -> solicitar -> aceitar -> pagar -> avaliar -> exportar comprovante -> ver Eco-Stats
- Deploy de producao funcional com variaveis de ambiente corretamente configuradas
- Documentacao final revisada e consistente com o estado real do sistema

3. Historias de Usuario do Sprint 8

O Product Owner apresentou cinco historias para este sprint, todas focadas em qualidade tecnica, seguranca e preparacao para producao. Nenhuma introduz funcionalidade nova fora do escopo atual — todas fecham lacunas existentes ou fortalecem o que ja foi entregue.

ID	Funcionalidade	Pontos	Prioridade	Sprint
US26	Endurecimento de Seguranca e Correcao de Vulnerabilidades	5 pontos	Alta	8
US27	Recuperacao de Senha e Integracao SMTP Final	3 pontos	Alta	8
US28	Visualizacao de Mapas com Leaflet.js	3 pontos	Media	8
US29	Modularizacao do Frontend e Limpeza de Codigo	3 pontos	Media	8
US30	Preparacao para Deploy e Documentacao Final	1 ponto	Alta	8
—	Total	15 pontos	—	—

4. Backlog Completo do Sprint 8

As tarefas abaixo foram definidas com base na análise crítica do SPRINT_STATUS.md, SEGURANCA.md, ROADMAP_FINAL.md e de todos os debitos tecnicos identificados ao longo do projeto. Cada tarefa e realista, fundamentada na documentacao e no estado atual do codigo.

4.1 Seguranca e Vulnerabilidades (US26)

ID	Titulo	Descricao Tecnica	Prioridade	Dificuldade	Responsavel
T01	Implementar Rate Limiting	Adicionar express-rate-limit nas rotas POST /usuarios/login, POST /usuarios e GET /api/* com limite de 20 req/min por IP	Alta	Baixa	Pedro Kafka
T02	Restringir CORS para Producao	Alterar configuracao de CORS de origin: '*' para o dominio real do frontend. Criar variavel de ambiente FRONTEND_URL e usa-la no cors()	Alta	Baixa	Rafael Machado
T03	Migrar JWT para HttpOnly Cookies	Alterar endpoint de login para retornar o token via Set-Cookie HttpOnly em vez de body JSON. Atualizar api.js para nao usar localStorage. Adicionar endpoint POST /api/logout para limpar o cookie	Alta	Media	Rafael Machado
T04	Implementar Protecao CSRF	Integrar csrf ou csurf no Express para proteger rotas mutantes (POST/PATCH/DELETE). Configurar frontend para incluir o token CSRF em todas as requisicoes de mutacao	Alta	Media	Ariane Archanjo
T05	Remover Console.log de Producao	Auditar todo o backend e frontend por console.log, console.error excessivos. Substituir por um logger condicional (ex: morgan em producao, ou flag NODE_ENV)	Media	Baixa	Matheus Sizanowski
T06	Validar Tipos de Upload mais Rigorosamente	Adicionar verificacao de magic bytes (cabecalho binario) nos uploads via	Media	Baixa	Pedro Kafka

		Multer para garantir que apenas imagens reais sejam aceitas, alem da validacao de extensao ja existente			
--	--	---	--	--	--

4.2 Recuperacao de Senha e SMTP (US27)

ID	Titulo	Descricao Tecnica	Prioridade	Dificuldade	Responsavel
T07	Finalizar Fluxo de Recuperacao de Senha	Ativar endpoint POST /api/usuarios/recuperar-senha: gerar token UUID, salvar na tabela verification_tokens com expires_at de 30 min, enviar e-mail com link de reset via Nodemailer	Alta	Media	Pedro Kafka
T08	Criar Tela de Redefinicao de Senha	Criar frontend/pages/redefinir-senha.html com formulario de nova senha + confirmacao. Consumir endpoint PATCH /api/usuarios/redefinir-senha com o token recebido por URL	Alta	Baixa	Ariane Archanjo
T09	Configurar e Testar SMTP com Nodemailer	Validar configuracao Nodemailer com SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS. Testar envio de e-mail de recuperacao e verificacao com servico gratuito (Gmail App Password ou Brevo)	Alta	Baixa	Matheus Sizanowski

4.3 Visualizacao de Mapas (US28)

ID	Titulo	Descricao Tecnica	Prioridade	Dificuldade	Responsavel
T10	Integrar Leaflet.js na Tela de Criar Carona	Adicionar mapa Leaflet.js em criar-carona.html com marcadores de Origem e Destino. Usar Nominatim (OpenStreetMap) para geocodificar os textos	Media	Media	Pedro Kafka

		digitados pelo usuário em coordenadas			
T11	Exibir Rota no Detalhe da Carona	Integrar Leaflet.js em carona.html para exibir a rota entre origem e destino com polyline. Exibir marcador especial para o ponto_encontro definido pelo motorista	Media	Media	Ariane Archanjo
T12	Filtro de Busca com Visualização no Mapa	Adicionar mapa opcional em buscar.html para exibir os pontos de origem das caronas encontradas. Clicar no marcador abre o detalhe da carona correspondente	Baixa	Media	Matheus Sizanowski

4.4 Modularização do Frontend (US29)

ID	Título	Descrição Técnica	Prioridade	Dificuldade	Responsável
T13	Quebrar chat-global.js em Módulos	Dividir o arquivo monolítico chat-global.js em: chat-state.js (estado), chat-api.js (chamadas fetch), chat-ui.js (renderização), chat-polling.js (setInterval). Garantir que a funcionalidade existente permaneça idêntica	Media	Media	Ariane Archanjo
T14	Modularizar notificações-global.js	Separar notificações-global.js em notificações-api.js e notificações-ui.js. Centralizar a lógica de polling em polling-manager.js compartilhado com o chat	Media	Baixa	Matheus Sizanowski
T15	Corrigir Race Condition na Lista de Espera	Adicionar SELECT ... FOR UPDATE no processarListaEspera() para garantir exclusividade de acesso quando o job roda. Alternativa: usar transação com	Alta	Media	Pedro Kafka

		BEGIN/COMMIT e lock no registro da carona			
T16	Padronizar Tratamento de Erros no Frontend	Revisar todos os catch() nos fetches do api.js para exibir mensagens de erro amigáveis via showAlert() em vez de alertas nativos do browser. Unificar mensagens em todos os idiomas (i18n)	Media	Baixa	Rafael Machado
T17	Melhorar Acessibilidade das Telas Críticas	Adicionar atributos ARIA (aria-label, aria-describedby, role) em botoes e formulários das telas login, cadastro, criar-carona e pagamento. Garantir que o fluxo principal seja navegável via teclado	Media	Baixa	Ariane Archanjo

4.5 Deploy e Documentação Final (US30)

ID	Título	Descrição Técnica	Prioridade	Dificuldade	Responsável
T18	Criar .env.example Completo	Atualizar .env.example com todas as variáveis dos Sprints 1-8: DATABASE_URL, JWT_SECRET, PIX_CHAVE, PIX_NOME, SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, EMAIL_FROM, FRONTEND_URL, NODE_ENV, PORT	Alta	Baixa	Matheus Sizanowski
T19	Validar Schema SQL e Migrations	Executar schema.sql do zero em banco limpo. Aplicar todas as migrations em ordem. Rodar data.sql de seeds. Confirmar que todas as tabelas, triggers e funções estão funcionando corretamente	Alta	Baixa	Pedro Kafka
T20	Atualizar README e Documentação Final	Revisar README.md com instruções de setup atualizadas para o	Alta	Baixa	Rafael Machado

		Sprint 8. Atualizar SPRINT_STATUS.md com status final de todas as funcionalidades. Revisar API.md com endpoints adicionados nos Sprints 7 e 8			
T21	Configurar Variaveis de Ambiente para Producao	Criar perfil de configuracao NODE_ENV=production. Garantir que console.log seja suprimido, CORS restrito e todos os secrets carregados do .env sem hardcode no codigo	Alta	Baixa	Rafael Machado
T22	Teste de Fluxo Completo pre-Entrega	Executar roteiro manual completo: cadastro -> verificacao e-mail -> login -> criar carona -> solicitar -> aceitar -> pagar -> concluir -> avaliar -> exportar PDF -> ver Eco-Stats. Registrar resultado de cada etapa	Alta	Baixa	Todos

5. Por quem sera feito

Membro	Responsabilidade Principal
Rafael Machado	Product Owner: restricao de CORS (T02), migracao JWT para cookies (T03), padronizacao de erros no frontend (T16), configuracao de variaveis de ambiente para producao (T21) e atualizacao da documentacao final (T20)
Ariane da Silva Archanjo	Scrum Master: implementacao de protecao CSRF (T04), visualizacao de rota no detalhe de carona (T11), modularizacao do chat-global.js (T13) e melhorias de acessibilidade (T17)
Pedro Kafka	Desenvolvimento: rate limiting (T01), validacao de uploads por magic bytes (T06), recuperacao de senha no backend (T07), integracao Leaflet em criar-carona (T10), correcao da race condition na lista de espera (T15), validacao do schema SQL (T19) e execucao do teste de fluxo completo (T22)
Matheus Sizanoski	Desenvolvimento: remocao de console.log de producao (T05), configuracao e teste SMTP Nodemailer (T09), filtro de busca com mapa (T12), modularizacao de notificacoes-global.js (T14) e criacao do .env.example completo (T18)

6. Como cada funcionalidade sera feita

US26 — Endurecimento de Seguranca (5 pontos)

Como equipe de desenvolvimento, queremos que o sistema esteja protegido contra as vulnerabilidades identificadas no documento SEGURANCA.md antes da entrega final.

Rate Limiting (T01)

Instalacao do pacote express-rate-limit. Configuracao de um limiter global e um limiter especifico para autenticao. O limiter de autenticao permitira no maximo 10 tentativas de login por IP em uma janela de 15 minutos, retornando HTTP 429 com mensagem clara. O limiter global protegera toda a API contra uso abusivo.

Migracao JWT para HttpOnly Cookies (T03)

O endpoint POST /api/usuarios/login sera alterado para definir o token como cookie HttpOnly, Secure (em producao), SameSite=Strict, com maxAge de 24 horas. O arquivo frontend/js/api.js sera atualizado para remover toda referencia ao localStorage do token. Um novo endpoint POST /api/logout limpaa o cookie. Todas as requisicoes autenticadas passarao automaticamente o cookie via credentials: 'include' no fetch.

Protecao CSRF (T04)

Integracao do middleware csrf no Express, configurado apos o middleware de cookie. O token CSRF sera exposto via endpoint GET /api/csrf-token e incluido automaticamente pelo frontend em todas as requisicoes mutantes via cabecalho X-CSRF-Token.

Criterio de Aceitacao	Responsavel	Status
Rate Limiting retorna 429 apos 10 tentativas de login em 15 min	Pedro Kafka	Planejado
Token JWT nao aparece no localStorage — apenas HttpOnly Cookie	Rafael Machado	Planejado
Requisicoes sem token CSRF valido retornam 403	Ariane Archanjo	Planejado
CORS rejeita origens nao autorizadas em NODE_ENV=production	Rafael Machado	Planejado
Upload de arquivo com extensao .jpg mas conteudo malicioso e bloqueado	Pedro Kafka	Planejado

US27 — Recuperacao de Senha e SMTP Final (3 pontos)

Como usuario, quero poder redefinir minha senha caso a esqueça, recebendo um link de recuperacao no meu e-mail institucional.

O fluxo completo sera: usuario acessa tela de login e clica em 'Esqueci minha senha'; insere o e-mail; o backend verifica se o e-mail existe, gera um UUID como token e o salva na tabela verification_tokens com expires_at de 30 minutos; o Nodemailer envia o e-mail com o link de redefinicao; o usuario clica no link, e redirecionado para redefinir-senha.html com o token na URL; o usuario define a nova senha; o backend valida o token, atualiza o hash da senha via bcrypt e invalida o token.

Criterio de Aceitacao	Status
E-mail de recuperacao e enviado em ate 30 segundos apos solicitacao	Planejado
Token expirado retorna erro 400 com orientacao de nova solicitacao	Planejado
Token ja utilizado retorna erro 400 — uso unico garantido	Planejado
Nova senha e salva como hash Bcrypt — nunca em texto plano	Planejado
Tela de redefinicao exibe feedback visual de sucesso e redireciona para login	Planejado

US28 — Visualizacao de Mapas com Leaflet.js (3 pontos)

Como usuario, quero visualizar no mapa a rota da carona e o ponto de encontro definido pelo motorista para entender melhor o trajeto antes de solicitar.

A implementacao usara Leaflet.js (ja citada no README como tecnologia de mapas), carregado via CDN. A geocodificacao dos enderecos textuais em coordenadas sera realizada via API gratuita do Nominatim (OpenStreetMap), sem custos ou chaves de API. O mapa sera inicializado com centro no Brasil e zoom adaptativo. Marcadores distintos identificarao Origem (verde), Destino (vermelho) e Ponto de Encontro (azul). A polyline da rota sera exibida via OSRM Route API (gratuita).

Criterio de Aceitacao	Status
Mapa exibido em criar-carona.html com marcadores ao digitar origem e destino	Planejado
Rota exibida em carona.html com marcadores de origem, destino e ponto de encontro	Planejado
Mapa responsivo — funciona corretamente em mobile (viewport estreito)	Planejado
Falha na geocodificacao exibe mensagem de erro sem quebrar a pagina	Planejado

US29 — Modularizacao do Frontend (3 pontos)

Como equipe de desenvolvimento, queremos que o codigo frontend esteja organizado em modulos coesos para facilitar manutencao, reuso e legibilidade tecnica.

O arquivo chat-global.js, identificado no SPRINT_STATUS.md como monolitico, sera dividido em quatro modulos: chat-state.js (estado da conversa ativa), chat-api.js (todas as chamadas fetch do chat), chat-ui.js (funcoes de renderizacao DOM) e chat-polling.js (gerenciamento do setInterval de polling). A mesma abordagem sera aplicada ao notificacoes-global.js. Um polling-manager.js centralizado unificara os intervalos e evitara multiplos setInterval concorrentes.

US30 — Preparacao para Deploy (1 ponto)

Como equipe, queremos garantir que o sistema possa ser configurado e executado em qualquer ambiente com o minimo de esforco, com documentacao clara e variaveis de ambiente completas.

O .env.example sera atualizado para incluir todas as variaveis do projeto. O README.md recebera a secao 'Configuracao de Producao' com instrucoes de NODE_ENV, build e deploy. O SPRINT_STATUS.md sera atualizado com status final de todas as funcionalidades. A API.md receberá os endpoints do Sprint 7 (eco-stats, verificar-email, reenviar-token, comprovante PDF) e Sprint 8 (logout, csrf-token, recuperar-senha, redefinir-senha).

7. Refatoracoes e Debitos Tecnicos

Esta secao documenta formalmente os debitos tecnicos identificados na analise do projeto e as acoes de refatoracao planejadas para o Sprint 8 Final.

7.1 Backend — Debitos Identificados

Debito Tecnico	Descricao do Problema	Acao de Correcao	Prioridade
Race Condition na Lista de Espera	Dois usuarios podem converter a mesma vaga simultaneamente quando o job processa no mesmo ciclo	SELECT ... FOR UPDATE em transacao BEGIN/COMMIT (T15)	Alta
Console.log em Producao	Logs excessivos expoe dados sensiveis e degradam performance em ambiente de producao	Logger condicional por NODE_ENV (T05)	Media
Falta de Rate Limiting	Rotas de login e API publica vulneraveis a ataques de forza bruta e DDoS basico	express-rate-limit (T01)	Alta
JWT no localStorage	Tokens JWT acessiveis por JavaScript expoe o sistema a ataques XSS	Migracao para HttpOnly Cookies (T03)	Alta
CORS Permissivo	origin: '*' permite que qualquer origem consuma a API — risco em producao	Restricao para FRONTEND_URL (T02)	Alta
Recuperacao de Senha sem SMTP	Rota de recuperacao existe no backend mas nao envia e-mail real por falta de credenciais SMTP	Configuracao Nodemailer (T09) + fluxo (T07)	Alta

7.2 Frontend — Debitos Identificados

Debito Tecnico	Descricao do Problema	Acao de Correcao	Prioridade
Arquivos JS Monoliticos	chat-global.js e notificacoes-global.js sao arquivos grandes com multiplas responsabilidades misturadas	Modularizacao em arquivos menores (T13, T14)	Media
Tratamento de Erros Inconsistente	Alguns catch() exibem alert() nativo do browser, outros usam showAlert(). Sem padrao definido	Padronizacao via showAlert() e i18n (T16)	Media
Acessibilidade Basica Ausente	Botoes e formularios criticos sem atributos ARIA. Fluxo nao navegavel via teclado	Adicionar aria-label, role e tabindex (T17)	Media
Ausencia de Mapas	Localizacoes das caronas sao apenas texto — sem visualizacao espacial para o usuario	Integracao Leaflet.js (T10, T11, T12)	Media

7.3 Banco de Dados — Melhorias

Item	Situacao Atual	Melhoria Planejada
Indices de Busca	Coluna status em caronas e solicitacoes_carona sem indice — queries de listagem podem ser lentas com volume	CREATE INDEX idx_caronas_status ON caronas(status); CREATE INDEX idx_solicitacoes_status ON solicitacoes_carona(status)
Tabela verification_tokens	Criada no Sprint 7 para tokens de email. Tokens expirados acumulam sem limpeza automatica	Adicionar job node-cron para DELETE FROM verification_tokens WHERE expires_at < NOW() executado diariamente
Indice de Busca por email	Busca por email em usuarios ocorre em queries criticas sem indice dedicado	CREATE INDEX idx_usuarios_email ON usuarios(email) — ja deve existir pelo UNIQUE, confirmar no schema

8. Testes do Sprint 8

Os testes são manuais e integrados ao desenvolvimento diário, com ciclo de validação no final da semana. Testes automatizados com Jest e Supertest complementam a cobertura dos módulos críticos.

Data	O que é testado	Como é testado	Responsável
19/05	Rate Limiting (T01)	Disparar 11 requisições de login em sequência rápida — verificar resposta 429 na 11ª com mensagem clara	Pedro Kafka
19/05	JWT em HttpOnly Cookie (T03)	Após login, abrir DevTools e verificar que o token não aparece em localStorage nem em Application > Cookies sem flag HttpOnly	Rafael Machado
19/05	CSRF Protection (T04)	Enviar POST /api/caronas sem o header X-CSRF-Token via curl — verificar resposta 403. Confirmar que o fluxo normal pelo frontend funciona corretamente	Ariane Archanjo
20/05	Recuperação de Senha (T07, T08, T09)	Solicitar recuperação de senha com e-mail válido -> verificar recebimento do e-mail -> clicar no link -> definir nova senha -> fazer login com nova senha. Testar token expirado e token já utilizado	Pedro Kafka
20/05	Leaflet.js — Mapas (T10, T11)	Criar carona e verificar que o mapa exibe marcadores de origem e destino. Acessar detalhe de carona e verificar polyline de rota. Testar em mobile (viewport 375px)	Ariane Archanjo
20/05	Modularização Frontend (T13, T14)	Verificar que o chat e notificações funcionam identicamente após modularização. Rodar ESLint no frontend e confirmar zero warnings	Matheus Sizanowski
21/05	Testes Automatizados — Cobertura	Rodar npm test e verificar cobertura $\geq 70\%$ nos serviços críticos (usuariosService, caronasService, pagamentosService). Confirmar que testes de integração do fluxo completo passam sem erros	Todos
21/05	Fluxo Completo pre-Entrega	Cadastro com e-mail institucional -> verificação por token -> login -> criar carona (com mapa) -> solicitar -> aceitar -> pagar -> concluir -> avaliar -> exportar PDF -> ver Eco-Stats ->	Todos

		recuperar senha. Product Owner valida cada etapa	
--	--	--	--

9. Publicacao e Deploy

Esta secao documenta o processo de preparacao e execucao do deploy final do UniCaronas para um ambiente de producao ou de demonstracao academica.

9.1 Checklist de Deploy

#	Item do Checklist	Responsavel	Status
1	Variaveis de ambiente configuradas no servidor: DATABASE_URL, JWT_SECRET, SMTP_*, PIX_*, FRONTEND_URL, NODE_ENV=production	Rafael Machado	Pendente
2	Schema SQL aplicado em banco de producao: schema.sql -> migrations -> data.sql (apenas seeds de demo)	Pedro Kafka	Pendente
3	CORS configurado com FRONTEND_URL real — sem wildcard *	Rafael Machado	Pendente
4	Rate Limiting ativo e testado em producao	Pedro Kafka	Pendente
5	SMTP validado: e-mail de verificacao e recuperacao de senha funcionando	Matheus Sizanowski	Pendente
6	Pasta backend/uploads/ com permissao de escrita no servidor	Pedro Kafka	Pendente
7	Frontend apontando para URL real do backend (sem localhost) em api.js	Ariane Archanjo	Pendente
8	npm start (modo producao) rodando sem erros — sem npm run dev	Rafael Machado	Pendente
9	Fluxo completo testado no ambiente de producao por pelo menos um membro da equipe	Todos	Pendente
10	README.md atualizado com URL de producao e instrucoes de acesso para avaliacao	Rafael Machado	Pendente

9.2 Tecnologias e Variaveis de Ambiente

Variavel de Ambiente	Descricao
PORT	Porta do servidor Express (padrao: 3000)
NODE_ENV	production em producao — ativa restricoes de CORS, suprime logs, habilita Secure em cookies
DATABASE_URL	String de conexao PostgreSQL: postgres://usuario:senha@host:5432/unicaronas
JWT_SECRET	Chave secreta para assinatura e verificacao de tokens JWT — minimo 32 caracteres aleatorios

PIX_CHAVE	Chave Pix do operador da plataforma (e-mail, CPF ou chave aleatoria)
PIX_NOME	Nome do recebedor exibido no payload Pix
SMTP_HOST	Servidor SMTP (ex: smtp.gmail.com, smtp-relay.brevo.com)
SMTP_PORT	Porta SMTP — 587 para TLS (STARTTLS) ou 465 para SSL
SMTP_USER	Conta de e-mail do remetente (usuario SMTP)
SMTP_PASS	Senha ou App Password da conta SMTP
EMAIL_FROM	Nome e endereco do remetente — ex: "UniCaronas" <contato@exemplo.com>
FRONTEND_URL	URL completa do frontend para configuracao do CORS (ex: https://unicaronas.vercel.app)

10. Riscos Tecnicos

Esta secao apresenta uma analise critica dos riscos tecnicos identificados para a fase final do projeto, com base na documentacao tecnica e no estado atual do sistema.

Risco	Probabilidade	Impacto	Mitigacao
Race Condition na Lista de Espera	Media	Alto	Implementar SELECT FOR UPDATE em transacao atomica (T15) antes do deploy final
Perda de Uploads no Servidor	Alta	Alto	Para producao academica: aceitar armazenamento local documentado. Para producao real: migrar para AWS S3 ou Cloudinary como proximo passo pos-entrega
SMTP com Limite de Envio	Media	Medio	Usar Brevo free tier (300 e-mails/dia) ou Gmail App Password. Documentar o limite no README
Quebra de Funcionalidade apos Modularizacao	Baixa	Alto	Executar testes manuais completos do chat e notificacoes apos refatoracao (T13, T14) antes de mergar na branch principal
Token CSRF quebrando Fluxo do Frontend	Media	Medio	Implementar e testar em ambiente de desenvolvimento antes de subir para producao. Garantir que api.js sempre inclui o token nas requisicoes mutantes
Falha na Geocodificacao Nominatim	Baixa	Baixo	Implementar fallback gracioso — se geocodificacao falhar, exibir mensagem de erro sem quebrar o formulario de criacao de carona
Divergencia entre Schema e Migrations	Baixa	Alto	Executar banco do zero com schema.sql antes do deploy final para validar que schema e migrations estao sincronizados (T19)

11. Tecnologias do Sprint 8

Tecnologia	Para que sera usada no Sprint 8
express-rate-limit	Rate limiting nas rotas de login, cadastro e API publica — protecao contra forca bruta (T01)
csrf (ou csrf)	Protecao CSRF para rotas mutantes POST/PATCH/DELETE (T04)
cookie-parser + HttpOnly Cookies	Armazenamento seguro do JWT token — substituicao do localStorage (T03)
Leaflet.js + Nominatim API	Visualizacao de mapas e geocodificacao de enderecos sem custo (T10, T11, T12)
OSRM Route API	Calculo e exibicao de rota entre origem e destino no mapa (T11)
Nodemailer + SMTP	Envio de e-mail de recuperacao de senha — finalizacao do fluxo pendente (T09)
Jest + Supertest	Ampliar cobertura de testes automatizados para $\geq 70\%$ nos servicos criticos (continuidade Sprint 7)
ESLint	Validacao de codigo modularizado do frontend — zero warnings (continuidade Sprint 7)
PostgreSQL — SELECT FOR UPDATE	Correcao da race condition no processamento da lista de espera (T15)
node-cron	Job de limpeza de tokens expirados na tabela verification_tokens (melhoria banco)
morgan (logger HTTP)	Substituicao dos console.log por logging profissional condicionado ao NODE_ENV (T05)

12. Visao Geral do Backlog — 2 Bimestre (Status Final)

O quadro abaixo consolida todos os itens do Roadmap com o status final apos o Sprint 8.

ID	Funcionalidade	SP	Prioridade	Sprint	Status
US19	Central de notificacoes em tempo real	5 SP	Alta	Sprint 6	Entregue
US20	E-mail de resumo semanal	3 SP	Media	Sprint 6	Entregue
US21	Indicador de digitacao no chat	2 SP	Media	Sprint 6	Entregue
US22	Validacao de e-mail por codigo/token	5 SP	Alta	Sprint 7	Entregue
US23	Refatoracao do codigo + testes	5 SP	Alta	Sprint 7	Entregue
US24	Dashboard de Impacto (Eco-Stats)	3 SP	Media	Sprint 7	Entregue
US25	Exportacao de comprovante em PDF	2 SP	Media	Sprint 7	Entregue
US26	Endurecimento de Seguranca e Vulnerabilidades	5 SP	Alta	Sprint 8	Planejado
US27	Recuperacao de Senha e SMTP Final	3 SP	Alta	Sprint 8	Planejado
US28	Visualizacao de Mapas com Leaflet.js	3 SP	Media	Sprint 8	Planejado
US29	Modularizacao do Frontend e Limpeza deCodigo	3 SP	Media	Sprint 8	Planejado
US30	Preparacao para Deploy e Documentacao Final	1 SP	Alta	Sprint 8	Planejado

13. Quando sera testado

Data	O que e testado	Como e testado
19/05	US26 — Seguranca e Vulnerabilidades	Rate limiting: disparar 11 requests de login -> verificar 429. Cookies: verificar ausencia de token no localStorage pos-login. CSRF: tentar POST sem token -> verificar 403. Uploads: enviar arquivo invalido -> verificar rejeicao
20/05	US27 — Recuperacao de Senha	Solicitar recuperacao -> verificar e-mail recebido -> clicar link -> definir nova senha -> login com nova senha. Testar token expirado e token ja utilizado. Testar com e-mail inexistente no sistema
20/05	US28 — Mapas Leaflet.js	Criar carona e verificar marcadores de origem/destino no mapa. Acessar detalhe de carona e verificar polyline. Testar responsividade em viewport 375px. Testar com endereco invalido para verificar fallback
20/05	US29 — Modularizacao Frontend	Verificar que chat e notificacoes funcionam identicamente apos modularizacao. Executar ESLint no frontend e confirmar zero warnings. Verificar que o polling nao cria multiplos intervals concorrentes
21/05	US30 — Deploy e Documentacao	Executar schema.sql em banco limpo -> verificar sucesso. Configurar variaveis de ambiente e rodar npm start -> verificar ausencia de erros. Validar README com instrucoes de setup
21/05	Fluxo Completo Final	Cadastro com e-mail institucional -> verificacao por token -> login -> criar carona com mapa -> solicitar -> aceitar -> pagar -> concluir -> avaliar -> exportar PDF -> ver Eco-Stats -> recuperar senha. Product Owner valida cada etapa e registra resultado

14. Quando vai para producao

Neste Sprint Final, 'ir para producao' representa a entrega academica completa com codigo estavel, testado, documentado e pronto para apresentacao ao corpo docente.

Data	O que acontece
21/05 (Qui)	Codigo final integrado na branch principal — todas as tasks do Sprint 8 concluidas, testes passando, ESLint sem warnings e documentacao atualizada
21/05 (Qui)	Apresentacao final do Sprint 8 — demonstracao das melhorias de seguranca, recuperacao de senha, mapas, modularizacao do frontend e checklist de deploy validado
21/05 (Qui)	Encerramento tecnico do projeto UniCaronas — documentacao final revisada e entregue junto ao repositorio GitHub

15. Conclusao Final

O Sprint 8 representa a conclusao de um ciclo de desenvolvimento que, ao longo de oito sprints progressivos, construiu uma plataforma de caronas universitarias tecnicamente madura e funcionalmente completa. Do sistema de autenticao com JWT e Bcrypt ao dashboard de impacto ambiental, da geracao de comprovantes em PDF a validacao automatica de e-mails institucionais — o UniCaronas demonstra a capacidade da equipe de entregar software real, com qualidade e consistencia.

Esta sprint final nao adiciona funcionalidades por adicionar; ela fortalece o que foi construido. As correcoes de seguranca (Rate Limiting, HttpOnly Cookies, CSRF, CORS restrito) transformam um sistema funcional em um sistema responsavel. A recuperacao de senha funcional fecha uma das lacunas mais impactantes para o usuario final. A integracao com Leaflet.js adiciona a camada visual que transforma enderecos em texto em rotas reais no mapa. A modularizacao do frontend eleva a qualidade tecnica do codigo a um patamar sustentavel.

O impacto academico do projeto e tangivel: a equipe aplicou na pratica conceitos de Engenharia de Software, Seguranca de Sistemas, Banco de Dados Relacional, Arquitetura de APIs RESTful, Metodologia Agil com Scrum e experiencia de usuario. O UniCaronas esta tecnicamente pronto para ser publicado, apresentado e utilizado como referencia de desenvolvimento de software por equipes estudantis.

Os proximos passos naturais apos a entrega — identificados no Roadmap — incluem a migracao para WebSockets (Socket.io) para substituir o polling, a integracao com gateway de pagamento real (Gerencianet/EFI) para confirmacao automatica do Pix, o armazenamento de uploads em nuvem (AWS S3) para persistencia em producao e a progressiva adocao de PWA para experiencia mobile nativa. Esses passos, embora fora do escopo academico atual, demonstram a clareza tecnica da equipe sobre o caminho de evolucao do produto.

Dimensao	Resultado do Projeto
Funcionalidades Core	100% entregues — autenticao, caronas, chat, pagamentos, avaliacoes, notificacoes
Funcionalidades Avancadas	Entregues — Eco-Stats, comprovante PDF, validacao por token, lista de espera automatica, recorrência
Seguranca	Endurecida no Sprint 8 — Rate Limiting, CSRF, HttpOnly Cookies, CORS restrito, validacao de uploads
Qualidade de Codigo	Servicos extraídos dos controllers, testes $\geq 70\%$, ESLint sem warnings, frontend modularizado
Documentacao	Completa — README, API.md, ARQUITETURA.md, BANCO_DE_DADOS.md, SEGURANCA.md, SPRINT_STATUS.md
Prontidao para Deploy	Checklist completo validado — variaveis de ambiente, schema SQL, CORS, SMTP, uploads, logs

UniCaronas — Sprint 8 Final | Engenharia de Software 3ESCN | 2026

Ariane Archanjo . Matheus Sizanowski . Pedro Kafka . Rafael Machado